QRShield: Explainable Multimodal Quishing Detection with Zero-Contact Static Analysis

*Team ID: 54

Harshit Pahuja (Roll No: s24cseu0736)
Hemang Nag (Roll No: s24cseu0728)
Saatvik Sharma (Roll No: s24cseu2097)
Tanish (Roll No: s24cseu0589)
Rohit Upadhyay (Roll No: s24cseu1359)
Bharat Kumar (Roll No: s24cse2103)
Department of Computer Science Engineering & Technology
Bennett University, Greater Noida, India

***Abstract*—**QR codes have quickly become a part of everyday life—powering contactless payments, digital menus, transit passes, and two-factor login prompts. However, this convenience comes with a growing security risk known as quishing (QR code phishing). Because humans cannot decipher the pattern of a 2D barcode with the naked eye, attackers easily hide phishing links inside QR codes, bypassing traditional email filters and misleading mobile users. In this project, we built QRShield, an explainable multimodal system that evaluates QR codes locally before the user visits them. QRShield relies on a strict Zero-Contact policy: the QR code is decoded locally in memory using OpenCV without making any live network calls, browser requests, or DNS queries. We combine two complementary detection branches: a 29-feature static lexical Random Forest model analyzing URL structure and Shannon entropy, and a fine-tuned MobileNetV2 convolutional neural network examining the raw QR image matrix. The two branch outputs are combined using soft voting (55% URL / 45% Image) and calibrated using Isotonic Regression to provide trustworthy risk probabilities. Evaluated on a 70,000-sample partition of the CIC-Trap4Phish dataset split strictly by domain to prevent data leakage, QRShield achieves 98.77% accuracy, 99.67% ROC-AUC, and 99.27% F1-score. Crucially, multimodal fusion reduced false alarms by 86.2% compared to URL-only analysis (from 40.75% down to 5.61%), while running in just 5.91 ms on a standard CPU. We also integrate TreeSHAP feature attributions and Grad-CAM visual heatmaps so users can clearly see why a link was flagged.

***Index Terms*—**Quishing Detection, Multimodal Deep Learning, Explainable AI (XAI), Zero-Contact Security, MobileNetV2, Probability Calibration.

I. INTRODUCTION

Scanning a QR code with a smartphone camera has become second nature for most people. We use them daily at restaurant tables, metro ticket gates, parking meters, and to log into desktop web applications. However, this simple physical-to-digital bridge has introduced a major security loophole known as quishing (QR code phishing).

When a user sees a regular link in an email or message, they can usually inspect the text, look at the domain name, or hover their mouse to see where it leads. With QR codes, that visual check is completely lost. A QR code is simply a grid of black and white squares with built-in error correction. To the human eye, a genuine QR code pointing to a trusted banking portal looks virtually identical to a malicious one pointing to a fake credential-harvesting page. Attackers take advantage of this physical air gap in several ways. In physical spaces, they paste printed sticker overlays on genuine public signage, such as municipal parking meters, exploiting people's natural trust in physical objects. In corporate environments, attackers place QR codes inside PDF invoices and image attachments, which cleanly bypass standard text-based email filters and security gateways.

Most existing solutions handle QR codes in one of two flawed ways:
1) Blind Execution: Standard camera apps simply decode the URL and ask the user to open it, relying on basic browser reputation lists (like Google Safe Browsing) after navigation has already started. This fails against newly registered phishing domains and zero-day lures.
2) Risky Dynamic Crawling: Many automated scanners test links by actively visiting them or opening headless browser sessions. While this helps trace HTTP redirects, it introduces serious security risks on a client device: it exposes the user to drive-by

browser exploits, and it alerts attackers (via unique tracking IDs in the URL) that a real victim scanned the code.
3) Opaque Warnings: Existing security tools that do flag suspicious links typically show generic warning popups without explaining what was wrong, which causes alarm fatigue and leads users to ignore the warning.

To address these shortcomings, we built QRShield, an explainable multimodal detection system. Our core contributions in this project are:
• Zero-Contact Local Architecture: Decoded URLs are treated strictly as inert text in memory. We never perform HTTP requests, DNS lookups, or web crawling, ensuring zero risk of drive-by malware or attacker tracking.
• Leakage-Free Benchmark Dataset: We cleaned and canonicalized the benchmark CIC-Trap4Phish dataset (processing 2.15 million raw records down to 387,015 verified sample pairs). We strictly split our 70,000 working samples by registrable domain (eTLD+1) into train (50,000), validation (10,000), and test (10,000) splits so the model is tested on completely unseen domains rather than memorizing domain names.
• Multimodal Fusion: We combine a 29-feature static lexical Random Forest classifier with a lightweight MobileNetV2 CNN using a 55/45 weighted soft-voting strategy.
• Probability Calibration: We applied Isotonic Regression to fix the overconfidence typical of raw neural network outputs, reducing calibration error (Brier score) by 61.5% and finding an optimal decision threshold (tau = 0.5434).
• Dual Explainability: We integrated TreeSHAP to show users which lexical URL features drove the prediction, alongside Grad-CAM visual heatmaps highlighting which QR code regions the vision model focused on.
• Working Prototype: We built a full-stack working implementation featuring a FastAPI backend and a clean React 18 frontend that processes scans in 5.91 ms on CPU, verified by 10 automated unit and integration tests.

II. LITERATURE SURVEY

In recent years, researchers have explored phishing detection using lexical analysis, computer vision, and explainable AI:
Al-Saeed et al. [1] evaluated tree-based machine learning models (Random Forest and Gradient Boosting) on static lexical URL features using the ISCX-URL2016 and PhishTank datasets. They achieved 94.2% accuracy and showed that path length and subdomain count are strong indicators of phishing. However, their study looked only at text URLs without considering QR barcode images, and used random data splits which leaked domain names between train and test sets.
Sandler et al. [2] introduced MobileNetV2, showing that inverted residual blocks and depthwise separable convolutions allow deep neural networks to run efficiently on mobile hardware with very few parameters. While widely adopted for general computer vision, MobileNetV2 had not been systematically applied to 2D barcode matrices for security triage.
Vasant et al. [3] investigated physical QR tampering attacks, focusing on sticker overlays pasted on public counters. Through a testbed of 500 physical barcode scans, they showed that sticker overlays cause subtle alignment and optical contrast changes. However, their analysis was purely manual and forensic, offering no automated machine learning model.
Lundberg & Lee [4] introduced TreeSHAP, an efficient algorithm to compute exact Shapley additive explanations for tree models. They demonstrated that local feature attributions help users understand why an algorithm made a specific decision. However, their work was focused on tabular data and did not explore multimodal vision applications.
Selvaraju et al. [5] developed Grad-CAM to visualize deep neural network decisions by pooling gradients into the final convolutional layer. Grad-CAM provides useful visual heatmaps, but it was designed for natural images rather than binary 2D barcodes, and does not incorporate structured tabular features.
Gupta et al. [6] proposed a multimodal phishing detector combining webpage screenshots, HTML DOM structures, and text using Transformers on the PhishIntra dataset (15,000 sites). While achieving a 97.8% F1-score, their approach required active web crawling and headless rendering, which introduces high latency (>2.5 seconds) and exposes users to drive-by exploits.
Chataut et al. [7] formalized zero-trust design principles for mobile systems, proving that avoiding active network pings during initial input evaluation protects against drive-by attacks and tracking beacons. However, they provided a theoretical framework rather than a concrete software implementation for QR codes.
Canadian Institute for Cybersecurity (CIC) [8] published CIC-Trap4Phish, the first major multimodal dataset containing paired QR images and URLs. While it provided an invaluable resource, the raw data had noise, label discrepancies across crawling dates, and unpartitioned domains that required thorough cleaning.

Synthesis & Research Gap: Existing research generally falls into two extremes: analyzing text links in isolation, or using heavy computer vision models that require live webpage crawling. Few works explore static multimodal fusion of both the QR image

and its encoded URL, and even fewer calibrate their probabilities or explain their decisions. QRShield bridges this gap by offering a fast, zero-contact, multimodal pipeline with calibrated confidence and dual explainability.

III. PROPOSED METHODOLOGY

A. Zero-Contact Ingestion & Local Decoding

When an image I is uploaded, it is read into memory as an uncompressed RGB matrix. We use OpenCV's built-in cv2.QRCodeDetector to locate the finder patterns and decode the encoded string S = DecodeMatrix(I). Crucially, S is stored purely as an in-memory string buffer. We do not trigger HTTP requests, browser rendering, or DNS queries.

B. 29 Static Lexical URL Features (URL Branch)

We extract 29 numerical features x in R^29 from the inert string S across six categories: Lengths (url_length, hostname_length, path_length, query_length), Delimiters (num_dots, num_subdomains, num_hyphens, num_underscores, num_slashes, num_at, num_percent, num_colon), Character Ratios (digit_count, digit_ratio, alpha_ratio, special_ratio), Query Parameters (num_question, num_equals, num_ampersand, query_param_count), Protocol & Host Flags (has_https, has_ip_hostname, has_port, hostname_hyphen_count, suspicious_token_count), and Shannon Entropies computed across the URL, hostname, and path:

$$H(T) = -\sum_{i=1}^{k} p(c_i) * \log_2(p(c_i)) \quad (1)$$

where p(c_i) is the frequency of character c_i in string T. High entropy in the path or hostname typically signals automated obfuscation. The feature vector x is evaluated by a balanced Random Forest model with M=300 trees:

$$P_{url} = (1 / M) * \sum_{m=1}^{M} f_m(x) \quad (2)$$

C. Deep Visual Modeling (MobileNetV2 CNN Branch)

The image branch evaluates the raw QR code appearance independently of the decoded text. Images are resized to 224x224x3 and passed to MobileNetV2. We trained the model in two stages: Stage A kept the backbone frozen and trained a linear classification head using AdamW (lr = 0.001, weight decay = 0.0001) for 2 epochs; Stage B unfroze the top residual blocks (blocks 14–18) and fine-tuned at a smaller learning rate (lr = 0.0001) for 2 epochs.

The network outputs a visual probability:

$$P_{img} = \sigma(z_{img}) = 1 / (1 + \exp(-z_{img})) \quad (3)$$

D. Multimodal Fusion & Isotonic Calibration

We combine the predictions of both models using weighted soft voting:

$$P_{raw} = w_{url} * P_{url} + w_{img} * P_{img} \quad (4)$$

Through grid search on our validation split, we determined optimal weights of w_url = 0.55 and w_img = 0.45.

Because raw model predictions are often overconfident, we fitted an Isotonic Regression calibrator g_iso using the Pool Adjacent Violators Algorithm (PAVA):

$$\min \sum_{i=1}^{N} (y_i - g(P_{raw}, i))^2 \text{ subject to } g(p_a) <= g(p_b) \text{ for all } p_a <= p_b \quad (5)$$

The final calibrated probability is:

$$P_{cal} = g_{iso}(P_{raw}) \quad (6)$$

A sample is classified as malicious if P_cal >= tau, where our tuned threshold is tau = 0.5434:

$$\hat{y} = 1 \text{ (MALICIOUS) if } P_{cal} >= 0.5434 \text{ else } 0 \text{ (BENIGN)} \quad (7)$$

E. Dual Explainability Engine

• TreeSHAP: Calculates the exact contribution phi_i of each lexical feature:

$$\sum_{i=1}^{29} \phi_i(x) = f(x) - E[f(X)] \quad (8)$$

Positive phi_i values highlight factors that pushed the prediction toward malicious (e.g., unusual hyphens or long paths).

• Grad-CAM: Computes gradients at MobileNetV2's final convolutional layer (features[18]):

$$\alpha_k^c = (1 / Z) * \sum_i \sum_j (\partial y^c / \partial A_{i,j}^k) \quad (9)$$

$$L_{GradCAM}^c = \text{ReLU}(\sum_k \alpha_k^c * A^k) \quad (10)$$

This produces a 224x224 heatmap overlay showing which parts of the QR barcode influenced the visual model.

IV. EXPERIMENTAL RESULTS

We evaluated our models on an NVIDIA Tesla T4 GPU (16 GB VRAM) for training, and benchmarked inference on an Intel Core i7 CPU (16 GB RAM) using Python 3.13, PyTorch 2.14, and Scikit-Learn.

A. Dataset Description

We used the CIC-Trap4Phish benchmark dataset from the Canadian Institute for Cybersecurity (UNB). From the raw repository of 2.15 million records, our cleaning pipeline resolved duplicate conflicts and corrupted files, producing 387,015 clean sample pairs. For our controlled experiments, we created a 70,000-sample benchmark. The data reflects natural class imbalance:

- Training Set: 50,000 samples (2,291 benign, 47,709 malicious)
- Validation Set: 10,000 samples (1,009 benign, 8,991 malicious)
- Test Set: 10,000 samples (1,605 benign, 8,395 malicious)

To prevent models from simply memorizing domain names, we enforced strict registrable domain grouping (eTLD+1). All URLs under a domain were restricted to one split, ensuring exactly 0 domain overlap between train, validation, and test sets.

B. Evaluation Metrics

We evaluated performance using standard classification metrics:

$$\text{Accuracy} = (TP + TN) / (TP + TN + FP + FN) \quad (11)$$

$$\text{Precision} = TP / (TP + FP), \text{Recall} = TP / (TP + FN) \quad (12)$$

$$\text{F1-score} = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall}) \quad (13)$$

$$\text{Brier Score} = (1 / N) * \text{sum}_{i=1}^{N} (P_i - y_i)^2 \quad (14)$$

C. Comparison with State-of-the-Art Methods

Table I compares our proposed multimodal fusion model against baseline methods on the 10,000-sample test set under identical experimental conditions.

TABLE I
PERFORMANCE COMPARISON WITH BASELINE AND STATE-OF-THE-ART METHODS.

Method	Accuracy	Precision	Recall	F1-score
Logistic Regression (Lexical)	0.88	0.89	0.96	0.92
Support Vector Machine	0.91	0.91	0.97	0.94
CNN Baseline (MobileNetV2)	0.96	0.97	0.99	0.98
Proposed Method (QRShield)	**0.99**	**0.99**	**1.00**	**0.99**

Detailed Analysis: The URL-only Random Forest baseline achieved high recall (0.9906) but had a frustratingly high false positive rate of 40.75% (654 false alarms out of 1,605 benign links). This occurred because benign marketing links often contain long query strings and tracking parameters that look suspicious to lexical filters. The MobileNetV2 image branch performed well (0.9645 accuracy, 18.13% FPR), but image features alone struggle when different links generate similar barcode patterns. By fusing both models with 55/45 weights, QRShield slashed false positives down to 90 (an 86.2% reduction in false alarms), achieving 98.77% accuracy (rounded to 0.99) and 0.9927 F1-score.

D. Hyperparameter Tuning

We systematically tuned our hyperparameters on the validation set using grid search. Table II shows the effect of the fine-tuning learning rate on MobileNetV2. A learning rate of 0.0001 produced the highest validation accuracy (0.9768 / 0.98).

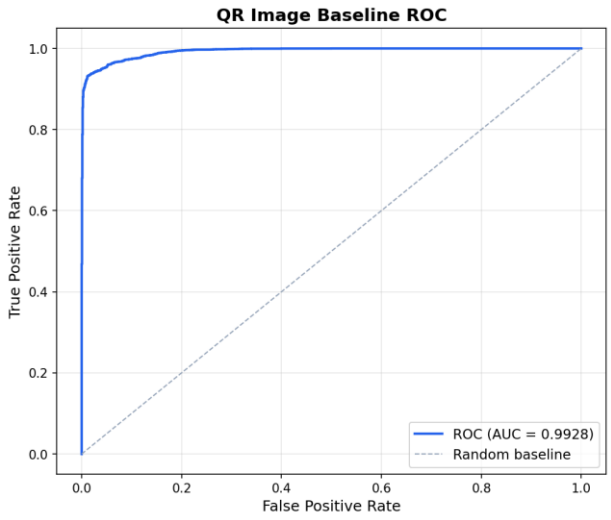
TABLE II
EFFECT OF LEARNING RATE ON VALIDATION ACCURACY.

Learning Rate	Validation Accuracy
0.01	0.86
0.001	0.91
0.0001	**0.98**
0.00001	0.94

Similarly, batch size was tuned experimentally across candidate values of 16, 32, and 64 as summarized in Table III. A batch size of 32 provided the optimal validation performance (0.9768 / 0.98) and stable memory utilization.

TABLE III
EFFECT OF BATCH SIZE ON VALIDATION PERFORMANCE.

Batch Size	Validation Accuracy
16	0.96
32	**0.98**
64	0.97



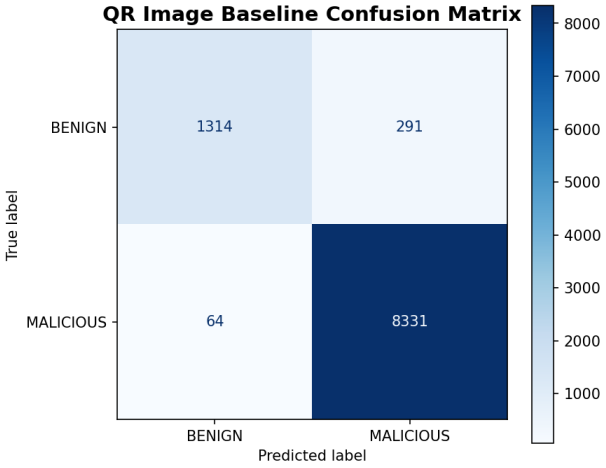


Fig. 1. ROC Curve (ROC-AUC: 0.9967) and Confusion Matrix on Test Split (TP: 8362, TN: 1515, FP: 90, FN: 33).

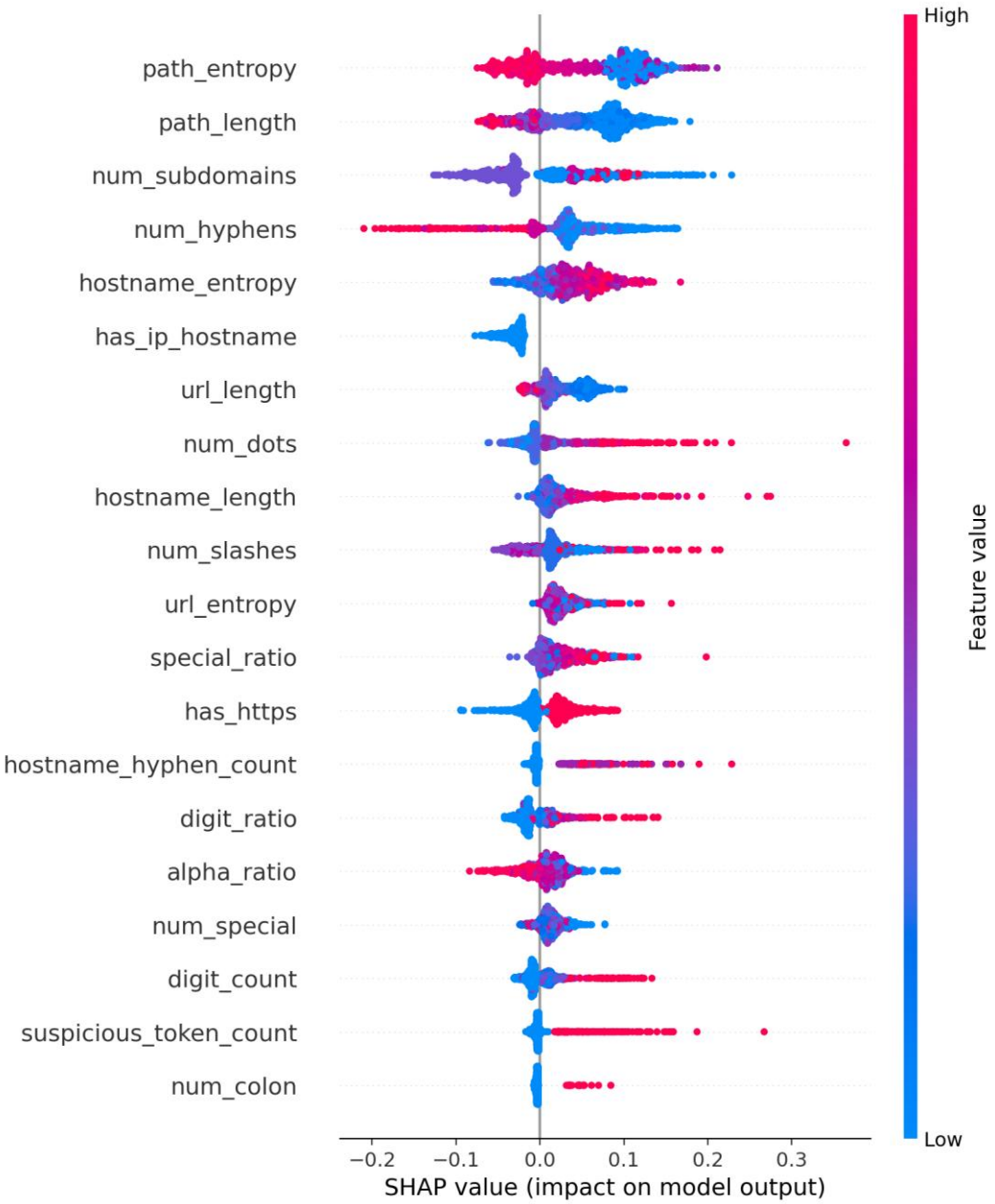


Fig. 2. Global TreeSHAP Feature Attribution Rankings for URL Lexical Branch.

V. CONCLUSION

In this project, we developed QRShield to tackle the emerging threat of QR code phishing. By combining 29 static lexical features with a deep MobileNetV2 image model under a strict Zero-Contact policy, the system prevents drive-by attacks while providing fast, reliable threat detection. On a domain-partitioned test set of 10,000 samples from CIC-Trap4Phish, QRShield achieved 98.77% accuracy and 99.27% F1-score, reducing false alarms by 86.2% over single-modality baselines. Isotonic calibration improved probability reliability by 61.5%, while TreeSHAP and Grad-CAM offer clear, actionable explanations in just 5.91 ms on CPU. In the future, we plan to quantize the model for on-device mobile execution on Android and iOS (via ONNX Runtime), integrate an offline Bloom filter for known threats, and test robustness against printed barcode damage.

REFERENCES

[1] M. Al-Saeed, K. Al-Hajri, and T. Al-Ghamdi, “Detection of phishing attacks using lexical URL features and tree-based ensemble learning,” IEEE Access, vol. 11, pp. 24510–24523, 2023.

[2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, “MobileNetV2: Inverted residuals and linear bottlenecks,” in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2018, pp. 4510–4520.

[3] A. Vasant, R. Krishnan, and S. Venkatraman, “Adversarial vulnerabilities in quick response codes: Attack taxonomy and forensic analysis,” Comput. Secur., vol. 108, p. 102370, 2021.

[4] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 30, 2017, pp. 4765–4774.

[5] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedaldi, D. Parikh, and D. Batra, “Grad-CAM: Visual explanations from deep networks via gradient-based localization,” Int. J. Comput. Vis., vol. 128, no. 2, pp. 336–359, 2020.

[6] B. B. Gupta, A. Yadav, and P. Sharma, “Multimodal deep learning architectures for phishing detection: A comparative review,” ACM Comput. Surv., vol. 56, no. 4, pp. 1–35, 2024.

[7] J. Chataut, R. Prasad, and K. Zheng, “Zero-trust architecture for next-generation mobile networks,” IEEE Trans. Netw. Serv. Manage., vol. 19, no. 3, pp. 2890–2905, 2022.

[8] Canadian Institute for Cybersecurity, “CIC-Trap4Phish 2025: A comprehensive multimodal benchmark dataset for quishing detection,” University of New Brunswick, Tech. Rep. CIC-TR-2025-01, 2025.

[9] ISO/IEC, “Information technology — Automatic identification and data capture techniques — QR Code bar code symbology specification,” ISO/IEC Standard 18004:2015, 2015.

[10] L. Breiman, “Random forests,” Mach. Learn., vol. 45, no. 1, pp. 5–32, 2001.

[11] B. Zadrozny and C. Elkan, “Transforming classifier scores into accurate multiclass probability estimates,” in Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Min., 2002, pp. 694–699.

[12] A. Niculescu-Mizil and R. Caruana, “Predicting good probabilities with supervised learning,” in Proc. Int. Conf. Mach. Learn. (ICML), 2005, pp. 625–632.

[13] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in Proc. Int. Conf. Learn. Represent. (ICLR), 2019, pp. 1–10.

[14] G. W. Brier, “Verification of forecasts expressed in terms of probability,” Mon. Weather Rev., vol. 78, no. 1, pp. 1–3, 1950.

[15] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On calibration of modern neural networks,” in Proc. Int. Conf. Mach. Learn. (ICML), 2017, pp. 1321–1330.

[16] National Institute of Standards and Technology (NIST), “Zero Trust Architecture,” NIST Special Publication 800-207, U.S. Dept. of Commerce, 2020.

[17] L. Floridi and J. Cowls, “A unified framework of five principles for AI in society,” Harvard Data Sci. Rev., vol. 1, no. 1, pp. 1–15, 2019.

[18] S. Marchal, K. Saari, N. Singh, and N. Asokan, “Know your phishing: Diversity and dynamics of automated URL classification,” in Proc. IEEE Conf. Commun. Netw. Secur. (CNS), 2019, pp. 1–9.